

# AI Fundamentals — Handout

A take-home summary of today's session. Six ideas, plain language, with places to read more.

## The big picture

Modern AI (ChatGPT, Claude, Gemini...) is built on one core idea: **predict the next chunk of text**, scaled up massively, with clever tricks layered on top. Here's how the six concepts fit together:

```
text → TOKENS → TRANSFORMER + ATTENTION → LLM → { RAG: open book }  
                                              { AGENT: takes action }  
                                              { MCP: standard tool socket }
```

## 0. How machines learn (the foundation)

Before AI could write, it learned to **predict**. The core flip behind all of it:

```
Classic programming: rules + data → answers  
Machine learning:   data + answers → rules (the "rules" = a pile of numbers)
```

- **Training** is one loop, run millions of times: **guess** → **measure the error** → **nudge the numbers to be less wrong** → **repeat**. (Like tuning a radio by ear.)
- **Recommendation systems** ("people like you also liked...") are the everyday example — and the bridge: a recommender predicts your next **click**; an LLM predicts the next **word**. Same kind of machine, different target, scaled up.
- What changed in the "AI boom" wasn't the recipe — it was the **scale** (billions of numbers instead of thousands) plus one architecture that trains fast: the **Transformer**.

**Mini-glossary.** **Data** = the examples you feed in (flows *through* the model). **Parameters** = the model's learned settings — they *are* the model (mostly called **weights**). "*Data passes through; parameters are the machine.*" A "70-billion-parameter model" = that many learned numbers.

## 1. Large Language Models (LLMs)

**What it is:** a program that read a huge amount of text and learned to predict the next word, one word at a time. It feeds its own output back in and repeats — that's why answers stream out word by word.

- "Large" = huge training data + billions/trillions of internal dials (**parameters**).
- Skills like translation, coding, and summarizing *emerged* from getting really good at next-word prediction — nobody programmed them directly.
- It has **no live memory** (each chat starts fresh; the app re-sends earlier messages) and its knowledge is **frozen** at a training cutoff date.
- **Hallucination:** it can state false things confidently, because its job is producing *plausible* text, not *verified* text.

**Everyday analogy:** phone autocomplete, scaled up about a million times.

## 2. Tokenization

**What it is:** before doing anything, the model chops text into small pieces called **tokens** and turns each into a number. The model only ever sees numbers.

```
"Tokenization is unbelievable!"
→ [ Token ][ is ][ un ][ believ ][ able ][ ! ]
→ [ 9145 ][318][1836][ 6667 ][1029 ][ 0 ]
```

- A token  $\approx \frac{3}{4}$  of a word (~4 characters). Common words = 1 token; rare words get split.
- This is why AI is billed **per token**, and why "context window" (how much text it can consider at once) is measured in tokens. ~750 words  $\approx$  1,000 tokens.
- It's also why models struggle to count letters ("how many R's in *strawberry*?") — they see chunks like **straw** + **berry**, not individual letters.

**Try it:** paste text into <https://platform.openai.com/tokenizer> and watch it split.

### 3. Transformers & Attention

**First, embeddings.** Before the Transformer does anything, each token is turned into an **embedding** — a list of numbers that acts as a point on a "meaning map," where similar things sit close together (so "cat" lands near "dog," far from "banana"). Meaning becomes geometry — famously, *king - man + woman  $\approx$  queen*. These start context-free ("bank" gets one vector regardless of sentence); attention's job is to refine them using context.

**What it is:** the **Transformer** is the design behind modern LLMs. Its key trick, **attention**, lets the model look at *all* the other words when processing each word, and decide which ones matter most.

"*The animal didn't cross the street because **it** was too tired.*" How do you know "it" = the animal? Because "tired" fits. The model figures it out the same way — "it" pays **attention** to "animal."

The animal ... because it was too tired  
          ↑                   |  
          └ strong attention ┘

- **Attention** = for each word, a weighted spotlight over all other words. Relevant words get a bright spotlight; irrelevant ones stay dim.
- This replaced older left-to-right models that "forgot" the start of long text. Attention handles long-range connections *and* runs fast in parallel — which made huge scale possible.
- Introduced in the 2017 paper "**Attention Is All You Need.**" ("GPT" = *Generative Pre-trained Transformer*.)

**One optional line of math:**  $\text{Attention} = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$  — in words: "match what each word is looking for (Query) against what others offer (Key), turn matches into percentages, then blend their content (Value)." You don't need this to use AI.

- **The other half of each block:** after attention shares info *between* words, a **feed-forward network** lets each word *process* what it gathered, on its own. Attention gathers the clues; the feed-forward step draws the conclusion. A model stacks dozens of these attention + feed-forward blocks (most of its parameters live in the feed-forward part).

### 3½. Multimodal (bonus)

**What it is:** the same Transformer isn't limited to text. **Multimodal** models also handle images, audio, and video — by chopping *any* input into tokens, the same trick from topic 2.

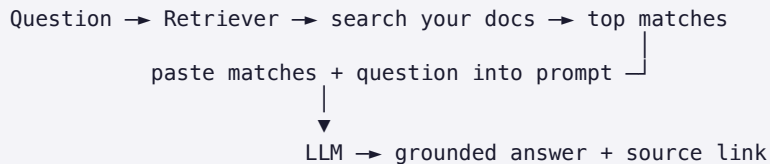
- An **image** is cut into a grid of small **patches**; each patch becomes a token, like a word-piece. **Audio** is sliced into time-slices, each a token.

- Once everything is tokens, the *same* attention runs — a word can attend to a patch of an image. That's how a model "looks at a photo and describes it," or turns your voice into a reply.

**Analogy:** tokenization is a universal adapter — text, pixels, and sound all become the same "token flour" the transformer can bake with.

## 4. RAG (Retrieval-Augmented Generation)

**What it is:** give the LLM an **open book** at question time. First *fetch* relevant documents, paste them into the prompt, then ask the model to answer using them.

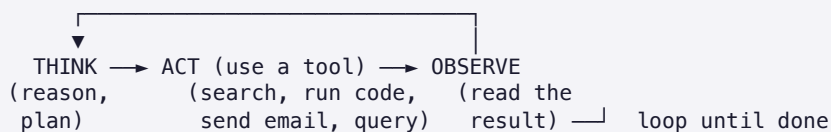


- Solves three LLM weaknesses: **stale knowledge**, **no access to your private data**, and **hallucination**.
- Search works by **meaning, not keywords** ("how do I get my money back?" finds the "refund policy" doc) — using those number-vectors (**embeddings**) from topic 3, stored in a **vector database**.
- The model itself is **not changed** — you only change what's in the prompt. Cheap to update: just add a document.

**Analogy:** closed-book exam (plain LLM) vs. open-book exam (RAG). Every "chat with your PDF" and most company help-bots are RAG.

## 5. Agents

**What it is:** an LLM that can *take actions* in a loop. It thinks, uses a **tool**, sees the result, and repeats until the goal is done. A plain LLM *talks*; an agent *does*.



- The LLM is the **brain**; **tools** are the **hands**. It doesn't really run code itself — it *requests* a tool, your system runs it, and feeds the result back.
- **Example:** "Book a table for 4 tomorrow." → search restaurants → check ratings → call the booking tool → confirm. Multiple steps, multiple tools, one goal.
- More autonomy = more risk, so consequential actions usually keep a **human in the loop**.

This think→act→observe pattern is often called **ReAct** (Reason + Act).

**Agent vs. workflow:** calling an LLM doesn't make a program an agent. If the *code* decides the steps and the model just fills them in (fixed path), that's a **workflow**. It's an **agent** only when the *model's* output decides the next action, in a loop. Litmus test: *does the LLM's output steer what happens next?* Rule of thumb: don't build an agent when a workflow will do.

## 6. MCP (Model Context Protocol)

**What it is:** a *standard plug* for connecting AI to tools and data — think **USB-C for AI**. Build a connector once per tool, and any MCP-aware AI app can use it.



- Before MCP, every app had to custom-build an integration for every tool. MCP replaces that mess with one shared standard.
- Ties back to **agents**: MCP is the standard way to *provide* an agent's tools. Agent = brain + loop; MCP = the sockets you plug the hands into.
- Introduced by Anthropic (late 2024); now widely adopted across the industry.

**Analogy:** before USB, every device had its own cable. MCP is the one port that fits.

### Want to go deeper? (beginner-friendly first)

- **Andrej Karpathy — *Intro to Large Language Models*** (1-hr video, best starting point): [https://www.youtube.com/watch?v=zjkBMFhNj\\_g](https://www.youtube.com/watch?v=zjkBMFhNj_g)
- **3Blue1Brown — *Neural Networks / Transformers series*** (beautiful visuals): <https://www.youtube.com/@3blue1brown>
- **Jay Alammar — *The Illustrated Transformer*** (the classic visual explainer): <https://jalammar.github.io/illustrated-transformer/>
- **Simon Willison — *Understanding GPT tokenizers***: <https://simonwillison.net/2023/Jun/8/gpt-tokenizers/>
- **Pinecone — *What is RAG?***: <https://www.pinecone.io/learn/retrieval-augmented-generation/>
- **Lilian Weng — *LLM-Powered Autonomous Agents***: <https://lilianweng.github.io/posts/2023-06-23-agent/>
- **Anthropic — *Model Context Protocol***: <https://modelcontextprotocol.io/>

**The key papers (for the curious):** *Attention Is All You Need* (2017, [arxiv.org/abs/1706.03762](https://arxiv.org/abs/1706.03762)) · *RAG* (2020, [arxiv.org/abs/2005.11401](https://arxiv.org/abs/2005.11401)) · *ReAct* (2022, [arxiv.org/abs/2210.03629](https://arxiv.org/abs/2210.03629))